

PSYC 51.17: Models of Language and Communication

Lecture 10: X-Hour Embeddings¹ Workshop

Week 3: Hands-On Dimensionality Reduction and
Word Vectors

PSYC 51.17: Models of Language and Communication

Learning Objectives

By the end of this session, you will:

1. Implement classic dimensionality reduction (LSA, LDA)
2. Train and analyze Word2Vec embeddings
3. Visualize high-dimensional embeddings using UMAP
4. Compare different embedding methods on real data
5. Understand semantic relationships captured by embeddings

Workshop format: Hands-on coding with the 20 Newsgroups dataset

Today's Agenda:

1. **Part 1:** Why embeddings? From sparse to dense representations
2. **Part 2:** LSA - Latent Semantic Analysis with SVD
3. **Part 3:** LDA - Latent Dirichlet Allocation for topic modeling
4. **Part 4:** Word2Vec - Neural word embeddings
5. **Part 5:** Visualizing embeddings with UMAP
6. **Part 6:** Comparing methods and document classification

Companion notebook:

Part 1: Why Embeddings?

The problem with sparse representations:

Last week (BoW, TF-IDF):

- High dimensional (vocab size)
- Sparse (mostly zeros)
- No semantic similarity
- "dog" and "puppy" are

Embeddings:

- Low dimensional (50-300 dims)
- Dense (all non-zero)
- Similar words cluster together

Sparse vs Dense: Concrete Comparison

```
1 # Sparse representation (one-hot / BoW)
2 # Vocabulary: [cat, dog, puppy, car, truck, vehicle]
3
4 cat_sparse = [1, 0, 0, 0, 0, 0] # 6 dimensions, 5 zeros
5 dog_sparse = [0, 1, 0, 0, 0, 0]
6 puppy_sparse = [0, 0, 1, 0, 0, 0]
7
8 # Cosine similarity: cat_dog = 0, dog_puppy = 0 (orthogonal)
```

Key insight: Dense embeddings capture that "dog" and "puppy" are semantically related, while sparse representations treat all words as equally different!

The Magic of Word Vectors

Famous example: king - man + woman = queen

Vector Arithmetic

Word embeddings capture semantic relationships as directions in space:

- Gender direction: woman - man
- Royalty direction: king - queen
- Pluralization: words - word

Key insight: Meaning encoded as geometry!

Vector Arithmetic: Step-by-Step Example

```
1 import numpy as np
2
3 # Pretend embeddings (simplified to 3D for illustration)
4 embeddings = {
5     'king': np.array([0.9, 0.8, 0.2]),
6     'queen': np.array([0.85, 0.75, 0.7]),
7     'man': np.array([0.7, 0.6, 0.1]),
8     'woman': np.array([0.65, 0.55, 0.6])
```

The math: Subtracting "man" removes the "male" component; adding "woman" adds the "female" component. The result is a "female royal" = queen!

Part 2: Latent Semantic Analysis (LSA)

Using SVD to find latent topics:

$$X \approx U_k \Sigma_k V_k^T$$

Algorithm:

1. Build TF-IDF matrix X
2. Apply Singular Value Decomposition
3. Keep top k dimensions

Interpretation:

- U : word-topic associations
- Σ : topic strengths
- V^T : doc-topic associations

LSA in Code

```
1 from sklearn.decomposition import TruncatedSVD
2 from sklearn.feature_extraction.text import TfidfVectorizer
3
4 # Build TF-IDF matrix
5 tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
6 tfidf_matrix = tfidf.fit_transform(documents)
7
8 # Apply LSA
9 lsa = TruncatedSVD(n_components=100, random_state=42)
10 doc_embeddings = lsa.fit_transform(tfidf_matrix)
11 word_embeddings = lsa.components_.T
```

Try it: Find similar words using cosine similarity!

LSA: Finding Similar Words

```
1 from sklearn.metrics.pairwise import cosine_similarity
2 import numpy as np
3
4 # Get vocabulary mapping
5 vocab = tfidf.get_feature_names_out()
6 word_to_idx = {word: i for i, word in enumerate(vocab)}
7
8 def find_similar_words(word, top_n=5):
9     """Find words with similar LSA embeddings."""
10    if word not in word_to_idx:
11        return f"'{word}' not in vocabulary"
12
13    idx = word_to_idx[word]
14    word_vec = word_embeddings[idx].reshape(1, -1)
15
16    # Compute similarities to all words
```

LSA: Finding Similar Words

```
17     sims = cosine_similarity(word_vec, word_embeddings)[0]
18
19     # Get top N (excluding the word itself)
20     top_indices = sims.argsort()[-top_n-1:-1][::-1]
21     return [(vocab[i], f"{sims[i]:.3f}") for i in top_indices]
22
23 print(find_similar_words("computer"))
24 # Output: [('software', 0.82), ('program', 0.79),
25 # ('system', 0.71), ('hardware', 0.68), ('disk', 0.65)]
```

Part 3: LDA for Topic Modeling

A probabilistic approach:

Generative Story

LDA imagines documents are created by:

1. Choosing a mixture of topics
2. For each word, picking a topic
3. Sampling a word from that topic

Key difference from LSA:

- Probabilistic interpretation
- Non-negative weights

LDA Example Output

- 1 Topic 0: hockey, game, team, player, season, nhl, play
- 2 Topic 1: space, nasa, launch, orbit, shuttle, satellite
- 3 Topic 2: computer, software, program, file, windows, system
- 4 Topic 3: medical, doctor, patient, disease, health, treatment
- 5 Topic 4: government, president, congress, law, political

Each document is a **mixture** of topics:

Document #42: 60% Space + 25% Computer + 15% Other

LDA: Complete Working Example

```
1 from sklearn.decomposition import LatentDirichletAllocation
2 from sklearn.feature_extraction.text import CountVectorizer
3
4 # 20 Newsgroups sample documents
5 documents = [
6     "The hockey team scored three goals in the game",
7     "NASA launched a new satellite into orbit",
8     "Install the software program on your computer",
9     "The doctor prescribed medicine for the patient",
10    # ... more documents
11 ]
12
13 # Step 1: Create bag-of-words matrix
14 vectorizer = CountVectorizer(max_features=1000, stop_words='english')
15 bow_matrix = vectorizer.fit_transform(documents)
```


LDA: Complete Working Example

```
16
17 # Step 2: Fit LDA
18 lda = LatentDirichletAllocation(n_components=5, random_state=42)
19 doc_topics = lda.fit_transform(bow_matrix)
20
21 # Step 3: Print topics
22 vocab = vectorizer.get_feature_names_out()
23 for topic_idx, topic in enumerate(lda.components_):
24     top_words = [vocab[i] for i in topic.argsort()[-7:]]
25     print(f"Topic {topic_idx}: {' '.join(top_words)}")
```

Part 4: Word2Vec

Learning embeddings from context:

Skip-gram:

Given target word, predict
context

"The **cat** sat on mat"

- cat → the, sat, on

CROW:

```
1 from gensim.models  
  Word2Vec  
2  
3 model = Word2Vec(  
4     16  
5     sentences=tokeniz  
    vector_size=100
```

Word2Vec: Semantic Similarity

```
1 # Find similar words
2 model.wv.most_similar('computer', topn=5)
3 # [('software', 0.82), ('program', 0.79), ('system', 0.75), ... ]
4
5 # Word analogies
6 model.wv.most_similar(
7     positive=['woman', 'king'],
8     negative=['man']
9 )
10 # [('queen', 0.71), ... ]
```

Hands-on Exercise

Try creating your own word analogies! What works? What fails?

Word2Vec: Exploring Analogies

```
1 # Analogies that typically WORK well:
2 model.wv.most_similar(positive=['paris', 'germany'], negative=
  ['france'])
3 # → 'berlin' (capital cities)
4
5 model.wv.most_similar(positive=['walking', 'swam'], negative=
  ['swimming'])
6 # → 'walked' (verb tenses)
7
8 model.wv.most_similar(positive=['bigger', 'colder'], negative=['big'])
```

Discussion: Why do some analogies work better than others?

Part 5: Visualizing with UMAP

Projecting 100D → 2D:

```
1 import umap  
2
```

UMAP advantages:

- Faster than t-SNE
- Preserves global structure
- Better cluster separation

What You Should See

When you visualize embeddings:

Sports cluster:

- hockey, baseball, player, team, game

Tech cluster:

- computer, software, program, windows

Space cluster:

- nasa, shuttle, orbit, launch,

Medical cluster:

- doctor, patient, hospital,

UMAP Visualization: Complete Code

```
1 import umap
2 import matplotlib.pyplot as plt
3
4 # Get word vectors for a subset of interesting words
5 words_to_plot = ['hockey', 'baseball', 'player', 'team', 'game',
6                 'nasa', 'shuttle', 'orbit', 'space', 'satellite',
7                 'computer', 'software', 'program', 'windows', 'disk',
8                 'doctor', 'patient', 'hospital', 'disease', 'treatment']
9
10 word_vectors = np.array([model.wv[w] for w in words_to_plot])
11
12 # Reduce to 2D with UMAP
13 reducer = umap.UMAP(n_neighbors=5, min_dist=0.3, metric='cosine')
14 embeddings_2d = reducer.fit_transform(word_vectors)
15
16 # Plot
```

UMAP Visualization: Complete Code

```
17 plt.figure(figsize=(12, 8))
18 plt.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1], alpha=0.7)
19 for i, word in enumerate(words_to_plot):
20     plt.annotate(word, (embeddings_2d[i, 0], embeddings_2d[i, 1]))
21 plt.title("Word Embeddings Visualized with UMAP")
22 plt.savefig("word_clusters.png")
```

Part 6: Comparing Methods

Method	Speed	Interpretability	Quality	Data Needed
LSA	Fast	Medium	Medium	Small-Medium
LDA	Medium	High	Medium	Medium
Word2Vec	Medium	Low	High	Large

Document Classification with Embeddings

Using embeddings as features:

```
1 def document_vector(doc, model):  
2     """Average word vectors for document."""  
3     tokens = preprocess(doc)  
4     vectors = [model.wv[w] for w in tokens if w in model.wv]  
5     return np.mean(vectors, axis=0) if vectors else np.zeros(100)  
6  
7 # Train classifier  
8 X_train = [document_vector(doc, w2v) for doc in train_docs]  
9 clf = LogisticRegression()
```

Compare to TF-IDF baseline!

Classification Comparison: Full Example

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.model_selection import cross_val_score
3
4 # Method 1: TF-IDF baseline
5 tfidf = TfidfVectorizer(max_features=5000)
6 X_tfidf = tfidf.fit_transform(train_docs)
7 clf_tfidf = LogisticRegression(max_iter=1000)
8 tfidf_scores = cross_val_score(clf_tfidf, X_tfidf, y_train, cv=5)
9
10 # Method 2: LSA embeddings
11 lsa = TruncatedSVD(n_components=100)
12 X_lsa = lsa.fit_transform(X_tfidf)
13 clf_lsa = LogisticRegression(max_iter=1000)
14 lsa_scores = cross_val_score(clf_lsa, X_lsa, y_train, cv=5)
```

Classification Comparison: Full Example

```
15
16 # Method 3: Word2Vec embeddings
17 X_w2v = np.array([document_vector(doc, model) for doc in train_docs])
18 clf_w2v = LogisticRegression(max_iter=1000)
19 w2v_scores = cross_val_score(clf_w2v, X_w2v, y_train, cv=5)
20
21 print(f"TF-IDF: {tfidf_scores.mean():.3f} (+/- {tfidf_scores.std():.3f})")
22 print(f"LSA: {lsa_scores.mean():.3f} (+/- {lsa_scores.std():.3f})")
23 print(f"Word2Vec: {w2v_scores.mean():.3f} (+/- {w2v_scores.std():.3f})")
```


2. **Different methods, different strengths:**

- LSA: Fast, linear, interpretable
- LDA: Probabilistic, topic-focused
- Word2Vec: Neural, best for similarity

3. **Visualization reveals structure - UMAP** shows semantic clusters

4. **Limitations:**

1. **Why does vector arithmetic work?** What does "king - man + woman" really mean geometrically?
2. **Bias in embeddings:** If Word2Vec learns from news articles, what biases might it capture?
3. **Window size matters:** What happens with window=2 vs window=10?
4. **Out-of-vocabulary problem:** How do you handle words not in your vocabulary?
5. **When to use what:** For a sentiment analysis task, would you choose LSA, LDA, or

For Assignment 2:

- Use embeddings to improve your classifier
- Compare at least 2 embedding methods
- Visualize your embeddings

Coming up in Lecture 11:

- Modern neural word embeddings
- GloVe and FastText
- Subword tokenization

Office hours: Available if you need help with the